ADR_011 API 設計慣例

> 狀態：已接受
> 日期：2026-10-14

背景

後端 API 現階段由測試工具與壓測腳本呼叫，日後要供會員網站、賣家後台、平台營運後台三個前端使用。需要在第一個 API 寫出來之前，訂好所有 API 共同遵守的慣例，避免各模組各自發明。

考慮過的選項與決定

主題	考慮過的選項	決定	理由
路徑的組織方式	依資源組織，由權限區分使用對象；或依使用對象分前綴	依使用對象分前綴：`/api/v1/public`、`/api/v1/member`、`/api/v1/seller`、`/api/v1/ops`；金流回呼另用 `/api/v1/webhooks`	三種使用者看到的資料形狀不同；每個前綴可套用不同的安全規則，不易漏掉權限檢查；日後要為各前端拆出專用的後端也容易
版本管理	放在路徑；放在標頭；放在媒體類型	放在路徑，例如 `/v1`	最直觀，日誌、閘道、快取都能直接辨識；標頭版本較「純粹」，但容易在除錯時被忽略

主題	考慮過的選項	決定	理由
錯誤格式	自訂格式；RFC 9457 Problem Details	Problem Details，另加平台的錯誤代碼 `code` 與 `traceId`	標準格式，Spring 原生支援；錯誤代碼讓前端不必解析文字；`traceId` 讓使用者回報問題時可直接追查
冪等	不支援；由請求內容判斷；以標頭傳遞冪等鍵	建立型操作以 `Idempotency-Key` 標頭傳遞冪等鍵	IETF 正在標準化的寫法，主流支付 API 也採用
更新權杖的傳遞	放在回應內容；放在 HttpOnly Cookie	HttpOnly、Secure、SameSite=Strict 的 Cookie	前端的 JavaScript 讀不到，即使被注入惡意程式也無法竊取效期長的更新權杖
規格與程式	先寫程式再產生規格；先寫規格再產生程式	先手寫 OpenAPI 規格，以工具產生 Java 介面，程式實作這些介面	規格是唯一的真相來源，程式無法偏離規格；前端日後可用同一份規格產生呼叫程式

其他慣例（命名、分頁、時間與金額格式）記錄於《設計_005_API規格》。

後果

- 好處：所有 API 行為一致；權限邊界在路徑上就看得出來；規格即文件

- 代價：同一份資料可能在不同前綴下各有一個 API（例如會員看結帳單、營運也看結帳單），需要分別維護；產生程式的工具有其限制，遇到時需調整規格寫法
- 重新檢討的條件：引入 API 閘道或為各前端拆出專用的後端時